# REVIEW 2: PROJECT REPORT

### Agentic Shell — Smart Archive Agent

| | |
|---|---|
| **Course / Project:** Operating Systems Project — Agentic Shell | **Project Title:** Smart Archive Agent |
| **Team Details:** Team 12 | **Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883) |

## Chapter 3: System Architecture & Design

### 3.1 High-Level Architecture

#### The User-to-Shell Loop: User Input → LLM Processing → Tool Call → System Execution → Feedback

Our agent is Agent #12 of the class Agentic Shell. In the full shell, the base shell's LLM reads what the user typed, picks one of our tools and fills in its arguments, and our code does the actual work on the system. We also wanted the agent to work on its own, without any LLM, so we wrote a rule-based planner that does the "LLM processing" step itself and produces the same tool calls. Either way, every call reaches the system through one checked entry point, execute_tool().

| Loop stage | Inside the Agentic Shell | When our agent runs on its own |
|---|---|---|
| **User Input** | The user types into the base shell. | The user types at our You: prompt (main.py). |
| **LLM Processing** | The shell's LLM reads our tool definitions (get_tools()) and picks tools by function calling. | intent.py and planner.py turn the sentence into a plan of tool calls, offline. |
| **Tool Call** | The shell calls execute_tool(name, arguments). | Each step of our plan goes to execute_tool(). |
| **System Execution** | tar, gzip, bzip2, xz and gpg run through subprocess.run() with argument lists. | Same. |
| **Feedback** | A result dictionary goes back to the LLM as the tool result. | We print the result: done lines, the output path, archive entries or the error. |

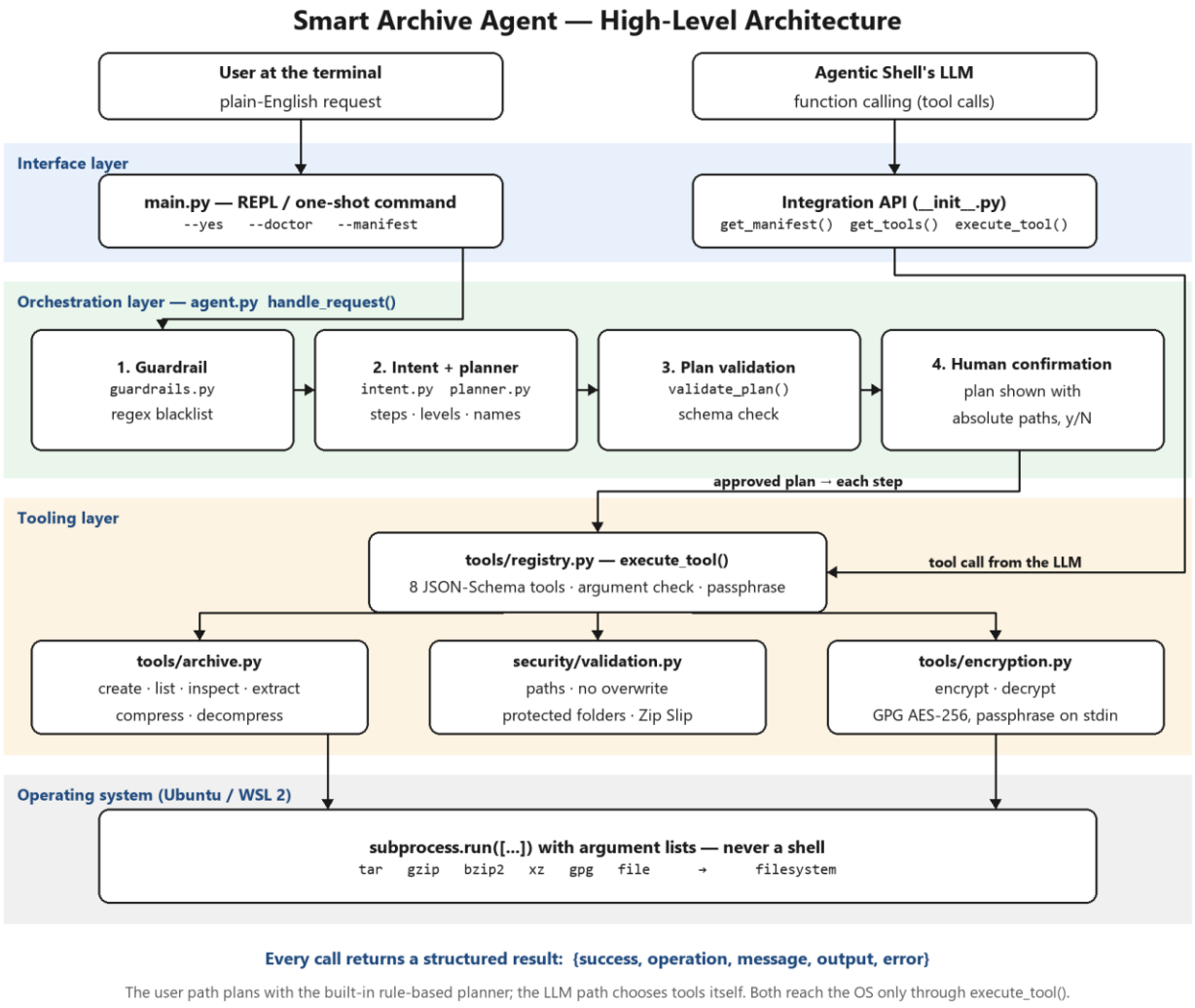



**Figure 3.1: High-Level System Architecture of the Smart Archive Agent.**

**How to read Figure 3.1.** There are two ways in, shown at the top: a user typing in the terminal, and the Agentic Shell's LLM. A typed request goes through our orchestration layer: the guardrail, the intent recognizer and planner, the plan check and the user's confirmation. A tool call from the LLM skips planning, because the LLM has already chosen the tool, and goes straight to the tool registry. Both paths then pass the same checks: execute_tool() compares the call with its JSON Schema, the security module checks every path again, and only then do our archive and encryption modules run the Linux tools. Every call returns the same kind of result.

## 3.2 Component Design

### 3.2.1 The Interface Layer: Custom REPL (Read-Eval-Print Loop) Environment

The interface is main.py together with the run() loop in agent.py. In interactive mode it prints a short banner and a You: prompt, shows the plan with full paths, asks y/N, asks for the passphrase without

showing it, and prints the result. We also added a one-shot mode for scripts. Ctrl+C or the end of input exits cleanly, and the exit code is 0 for success and 1 for failure.

```
python3 -m smart_archive_agent                          # interactive REPL
python3 -m smart_archive_agent "archive ./src"          # one request, asks before writing
python3 -m smart_archive_agent --yes "back up ./src"    # one request, no confirmation
python3 -m smart_archive_agent --doctor                 # checks tar, gzip, bzip2, xz, gpg,
file
python3 -m smart_archive_agent --manifest               # agent description for the shell's
router
```

Because our teacher will plug the agent into the class shell, the package itself is also part of the interface: __init__.py exports get_manifest(), get_tools(), execute_tool() and handle_request(), and agent.json tells the shell's router what our agent handles, what it does not, and what it needs installed.

### 3.2.2 The Orchestration Layer: Intent Routing

The template mentions LangChain, LlamaIndex or a raw MCP client for this layer. We decided to write the orchestration ourselves in plain Python (agent.py, handle_request()) instead of using a framework. We had three reasons: the agent has to install on our teacher's machine without extra packages, we wanted every decision to be predictable so we could reason about safety, and understanding open-ended language is already the job of the base shell's LLM. Each request goes through these stages in order:

| Stage | Module | What it does |
|---|---|---|
| **1. Guardrail** | security/guardrails.py | Checks the raw text against a regex blacklist of destructive commands before anything else. |
| **2. Intent recognition** | intent.py | Removes file paths from the text, then looks for whole-word verbs in a fixed order (decrypt, encrypt, extract, decompress, list, compress, archive). Requests that are not about archiving stop here. |
| **3. Planning** | planner.py | Builds the steps: folders are archived first; "heavily" → level max, "quickly" → fast; missing output names are chosen next to the source; an unencrypted archive we created is deleted after encryption. |
| **4. Plan validation** | planner.validate_plan() | Checks every step against the tool schemas; a clean-up step may only delete a file made earlier in the same plan. |
| **5. Confirmation and execution** | agent.run_plan() | Shows the plan, waits for y, runs the steps in order, and stops at the first failure, saying which steps finished. |

### 3.2.3 The Tooling Layer: Safe System Functions

Our agent offers eight archive functions as tools. In tools/registry.py each tool has a name, a description written so an LLM can choose it, and a JSON Schema for its parameters; this is the same shape an MCP

server uses to describe its tools. get_tools() returns these definitions in generic, OpenAI-compatible or Anthropic format, and execute_tool() runs one call. We have not wrapped them in an actual MCP server yet; we list that as future work. System monitoring, network configuration and other file tasks belong to other agents in the class shell.

| Tool | Python function | Linux backend | Key arguments |
|---|---|---|---|
| **create_archive** | archive.create_archive() | tar with gzip / bzip2 / xz (tar -I), or Python zipfile | source_path, output_path, compression, level |
| **list_archive** | archive.list_archive() | tar -tf, zipfile | archive_path |
| **inspect_archive** | archive.inspect_archive() | file -b, tar -tf, zipfile | archive_path |
| **extract_archive** | archive.extract_archive() | Python tarfile / zipfile after checking every member | archive_path, destination_path |
| **compress_file** | archive.compress_file() | gzip, bzip2, xz | source_path, output_path, algorithm, level |
| **decompress_file** | archive.decompress_file() | gzip -d, bzip2 -d, xz -d | source_path, output_path |
| **encrypt_file** | encryption.encrypt_file() | gpg --symmetric --cipher-algo AES256 | source_path, output_path (+ passphrase) |
| **decrypt_file** | encryption.decrypt_file() | gpg --decrypt | source_path, output_path (+ passphrase) |

Compression levels: fast = gzip -1 / xz -1, normal = gzip -6 / xz -6, max = gzip -9 / bzip2 -9 / xz -9e.

### 3.2.4 Result Handling & Diagnostics

Every tool and every request returns one dictionary: success (true or false), operation, a message for people, output (the path that was created), plus entries for listings, steps and plan for multi-step requests, missing when a detail could not be found, and error when something failed. execute_tool() never raises an exception; errors inside a tool come back as success = false with a message, so a failing operation cannot crash our REPL or the base shell. The same dictionary can be passed back to an LLM with json.dumps().

## 3.3 Security & Guardrail Framework

### 3.3.1 The "Human-in-the-Loop" Validation Mechanism

Nothing is written until the user has seen and approved the whole plan. Before any step that writes a file, our agent prints every step with full source and destination paths and waits for y; any other answer cancels with "Nothing was changed". Read-only requests (list, inspect) run without asking. The passphrase for encryption and decryption is always typed by the user at a hidden prompt (twice when encrypting), never supplied by an LLM. When the agent runs inside the base shell, handle_request(confirm=…, passphrase=…) lets the shell show the plan and collect the passphrase in its own way.

### 3.3.2 Command Sanitization and Regex-Based Blacklisting of Destructive Commands

User text is never run as a command. We start every tool with subprocess.run() and a list of arguments (never shell=True), so characters like ; | $( ) are treated as plain text. On top of that, we check the raw request against a regex blacklist before planning, so a destructive request is refused completely instead of being half carried out:

| Pattern (examples) | Refused as |
|---|---|
| **rm -rf …, rm -fr …** | recursive forced delete |
| **mkfs, mkfs.ext4** | filesystem formatting |
| **dd if=…** | raw disk write |
| **:(){ :\|:& };:** | fork bomb |
| **> /dev/sda** | raw block-device overwrite |
| **chmod -R 777 /** | world-writable root filesystem |
| **shutdown, reboot, poweroff, halt** | system power control |
| **format / wipe / erase … disk, drive, everything** | disk wipe request |
| **delete everything, delete all files** | mass deletion request |
| **; rm, \| bash, `…`, $( … )** | shell command injection |
| **a bare / as a path** | operation on the whole root filesystem |

Inside every tool there are more checks, whichever way the call arrived:

- **Protected locations:** we never write into /bin, /boot, /dev, /etc, /lib, /lib64, /proc, /sbin, /sys, /usr or /var/lib.
- **No overwrites:** an existing file is never replaced; source and output must be different; an archive cannot be written inside the folder being archived.
- **Zip Slip prevention:** every member of an archive must stay inside the destination; symbolic and hard links inside tar archives are refused; the destination folder is only created after the whole archive has passed.
- **Schema check of tool calls:** unknown tools, missing or extra arguments, wrong types and invalid values (for example level = "ultra") are rejected before anything runs.
- **Least privilege:** the agent runs as a normal user and never uses sudo.

# Chapter 4: Implementation Methodology

## 4.1 Development Environment & Tech Stack

| Item | What we used |
|---|---|
| **Programming Language** | Python. We wrote the code for Python 3.8 or newer using only the standard library (subprocess, tarfile, zipfile, getpass, json, re, pathlib); we developed and tested it with Python 3.14.4. |
| **Operating System** | Ubuntu 26.04.1 LTS on Windows Subsystem for Linux 2 (Windows 11). It is meant for any Ubuntu or Debian machine. |
| **LLM Provider** | Provided by the base Agentic Shell, which owns the LLM. get_tools() gives our tool definitions in OpenAI-compatible format (which local Ollama also accepts) and Anthropic Claude format. Our agent does not call an LLM itself and needs no API key; offline requests use our own planner. |
| **SDKs** | None needed. We did not use the MCP Python SDK; our tool definitions follow the same JSON Schema structure, so we can add an MCP server later without changing the tools. |
| **System Utilities** | GNU tar 1.35, gzip, bzip2, XZ Utils 5.8.3, GnuPG 2.4.8, file. |
| **Testing** | pytest 9.1.1 (58 automated tests) and our script benchmarks/evaluate.py for accuracy, speed and error recovery. |
| **Packaging & Integration** | pyproject.toml (pip install from GitHub, smart-archive-agent command), the agent.json manifest, INTEGRATION.md and a runnable integration example. |
| **Tools** | Visual Studio Code with the WSL extension, Git and GitHub. |

## 4.2 Step-by-Step Implementation

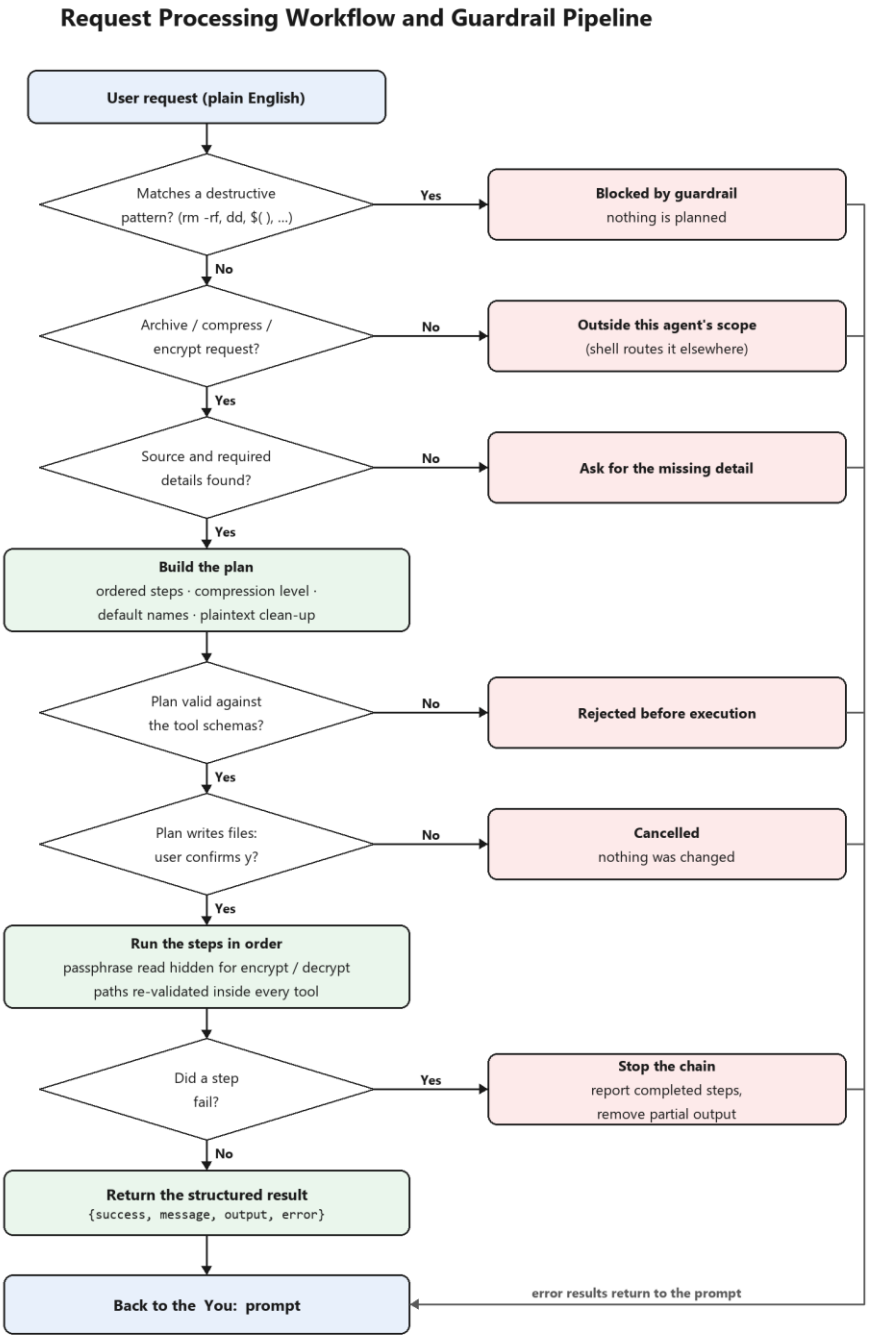



**Figure 4.1: End-to-End Request Processing Workflow and Guardrail Pipeline.**

Figure 4.1 follows one request through our agent. Every decision has a safe way out on the right (blocked, out of scope, missing detail, rejected, cancelled, or a failed step), which returns an error and goes back to the prompt without changing any file. We built the implementation in four steps.

**Step 1: Setting up the base CLI prompt loop**

We started with main.py, which reads the command line (interactive, one-shot, --yes, --doctor, --manifest, --version), and the run() loop in agent.py: read a line, pass it to handle_request() with a terminal confirmation and a hidden passphrase prompt, print the result with print_result(), and repeat until exit, Ctrl+C or the end of input. We added --doctor so that on any lab machine we can see at once which Linux tool is missing and the apt install command for it.

**Step 2: Building the MCP-style tool layer to expose safe system functions**

Next we wrote the eight operations in archive.py and encryption.py as plain Python functions that check their own paths and return result dictionaries. Then, in tools/registry.py, we described each one for function calling (TOOL_SCHEMAS) and wrote execute_tool(), the only function that actually runs a tool. The template's examples, list_directory and get_system_stats, match list_archive and inspect_archive in our domain: they read an archive without changing anything. We also added an option to prefix our tool names (smart_archive__create_archive) so they stay unique among the 35 agents.

**Step 3: Prompt engineering for exact tool matching and JSON format enforcement**

For the LLM path we wrote each tool and parameter description so that a model can pick the right tool and values from everyday words, and we used JSON Schema to force the format:

```
"level": {"type": "string", "enum": ["fast", "normal", "max"],
          "description": "Compression effort. Use 'max' when the user says heavily, maximum
                          or as small as possible; 'fast' for quick; otherwise 'normal'."}
"required": ["source_path", "output_path"],   "additionalProperties": false
```

check_arguments() applies the schema to every call (unknown tool, missing or extra argument, wrong type, value outside the enum), and execute_tool() also accepts the JSON string that LLM APIs return. For our offline planner, intent.py matches whole words in a fixed order after removing the file paths from the text, so "zip" is not found inside "gzip" and "tar" is not found inside "backup.tar.xz". We measured how well this works on phrasings we never tuned on (Review 3, Section 5.2.1).

**Step 4: Implementing the command execution wrapper and output formatter**

All Linux tools run through one wrapper that passes an argument list, captures the output, uses a 600-second timeout, closes standard input and turns failures into readable errors:

```
subprocess.run(["tar", "-I", "xz -9e -T0", "-cf", output, "-C", parent, name],
               check=True, capture_output=True, text=True,
               timeout=600, stdin=subprocess.DEVNULL)

# passphrase is sent on standard input, never as an argument
gpg --batch --yes --no-symkey-cache --pinentry-mode loopback \
    --passphrase-fd 0 --symmetric --cipher-algo AES256 \
    --output <file>.gpg <file>
```

run_plan() runs the steps in order, remembers which files each step created (so the clean-up step can only delete those), stops at the first failure and removes any half-written output. print_result() prints the result in the terminal as done lines, the output path, archive entries and errors. We report how we tested all of this, and the results, in Review 3 (Chapter 5).

# References


[1] Python Software Foundation, "subprocess — Subprocess management," Python 3 documentation. Available: https://docs.python.org/3/library/subprocess.html

[2] Free Software Foundation, "GNU tar: an archiver tool," manual for GNU tar 1.35. Available: https://www.gnu.org/software/tar/manual/

[3] W. Koch et al., "The GNU Privacy Guard (GnuPG) Documentation." Available: https://gnupg.org/documentation/

[4] JSON Schema, "JSON Schema specification." Available: https://json-schema.org/

[5] OpenAI, "Function calling guide." Available: https://platform.openai.com/docs/guides/function-calling

[6] Anthropic, "Model Context Protocol specification." Available: https://modelcontextprotocol.io/

[7] OWASP Foundation, "Path Traversal." Available: https://owasp.org/www-community/attacks/Path_Traversal

[8] Python Software Foundation, PEP 706, "Filter for tarfile.extractall." Available: https://peps.python.org/pep-0706/

# Appendix B: Implementation Evidence — Review 2 Stage

This appendix shows the implementation from Chapters 3 and 4 as it is in our repository: the commits we made while building it, the size of each module, the most important parts of the code, and real output from running it.

## B.1 Our Implementation Commits (Stage 2)

We rebuilt the agent one module at a time and committed each module separately: validation first, then the archive tools, encryption, the intent router, the agent loop and the command line, followed by the tests, the design diagrams and the documentation.

**Table B.1: Our Stage 2 commits (from git log).**

| Commit | Author | What we changed | Size |
|---|---|---|---|
| **7856c51** | Kush Aghera | updated validation logic | 2 files, +30 / −225 lines |
| **000e677** | Kush Aghera | updated archive tools | 2 files, +135 / −327 lines |
| **70aa599** | Kush Aghera | updated encryption tools | 1 file, +38 / −176 lines |
| **d14e9e3** | Kush Aghera | updated intent router | 1 file, +61 / −209 lines |
| **c39ecda** | Kush Aghera | updated agent loop | 1 file, +58 / −547 lines |
| **a7a5f08** | Kush Aghera | updated main cli | 2 files, +19 / −253 lines |
| **6d403e4** | Kush Aghera | updated project dependencies | 2 files, +5 / −4 lines |
| **87ab7ae** | Kush Aghera | added test cases | 2 files, +161 / −60 lines |
| **018788e** | Kush Aghera | cleaned old tests | 4 files, +0 / −329 lines |
| **a1fff6e** | Kush Aghera | added design diagrams | 2 files (binary: .docx / .pdf / .png) |
| **9114e3b** | Kush Aghera | added project memory | 1 file, +39 / −0 lines |
| **bfe5279** | Kush Aghera | added review templates | 3 files (binary: .docx / .pdf / .png) |
| **c97944d** | Kush Aghera | updated project readme | 1 file, +88 / −54 lines |
| **67f6a44** | Kush Aghera | added reference guide | 1 file, +122 / −0 lines |
| **c8f53ca** | Kush Aghera | polished project readme | 1 file, +138 / −93 lines |

## B.2 Size of Each Module

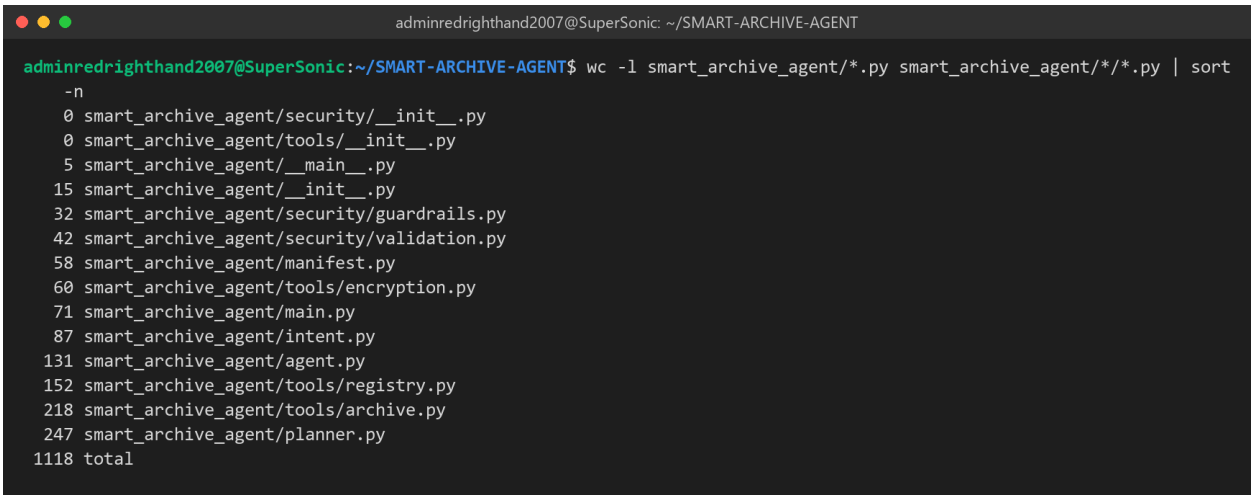



**Figure B.1: Lines of code in each module of the smart_archive_agent package (1,118 lines in total).**

## B.3 Key Parts of Our Code

These excerpts are copied straight from our repository, with their real line numbers.

### Guardrail: regex blacklist checked before any planning (Section 3.3.2)

```
  6  DESTRUCTIVE_PATTERNS = [
  7      (re.compile(r"\brm\s+-[a-z]*r[a-z]*f|\brm\s+-[a-z]*f[a-z]*r", re.I), "recursive forced delete (rm -
rf)"),
  8      (re.compile(r"\bmkfs(\.\w+)?\b", re.I), "filesystem formatting (mkfs)"),
  9      (re.compile(r"\bdd\s+if=", re.I), "raw disk write (dd)"),
 10      (re.compile(r":\(\)\s*\{\s*:\|:&\s*\};:"), "fork bomb"),
 11      (re.compile(r">\s*/dev/(sd|nvme|hd)\w*", re.I), "raw block-device overwrite"),
 12      (re.compile(r"\bchmod\s+(-R\s+)?777\s+/(\s|$)", re.I), "world-writable root filesystem"),
 13      (re.compile(r"\b(shutdown|reboot|poweroff|halt)\b", re.I), "system power control"),
 14      (re.compile(r"\b(format|wipe|erase|destroy)\b.*\b(drive|disk|partition|system|everything)\b", re.I),
 15       "disk wipe request"),
 16      (re.compile(r"\bdelete\s+(everything|all\s+files|all\s+my\s+files)\b", re.I), "mass deletion
request"),
 17      (re.compile(r"(?:^|\s)/\*?(?:\s|$)"), "operation on the whole root filesystem (/)"),
 18      (re.compile(r"[;&|`]\s*(rm|sudo|curl|wget|bash|sh)\b|\$\(", re.I), "shell command injection"),
 19  ]
 20
 21  # Output locations the agent will never write into, even if the user asks.
 22  # /root is deliberately absent: it is the home folder when the shell runs as root.
 23  PROTECTED_DIRS = ("/bin", "/boot", "/dev", "/etc", "/lib", "/lib64", "/proc",
 24                    "/sbin", "/sys", "/usr", "/var/lib")
 25
 26
 27  def check_request(text: str) -> str | None:
 28      """Return a refusal reason if the request matches a destructive pattern."""
 29      for pattern, reason in DESTRUCTIVE_PATTERNS:
 30          if pattern.search(text):
 31              return reason
 32      return None
```

*Listing B.1: Our destructive-command patterns and check_request(). Source: smart_archive_agent/security/guardrails.py, lines 6–32.*

### Planner: chaining the archive, compression level and encryption (Section 3.2.2)

```
166      wants_encrypt = ENCRYPT_RE.search(words) is not None
167      named_archive = named_out is not None and archive_format_for(named_out) is not None
168      wants_archive = (op == "create_archive" or source_abs.is_dir() or named_archive
169                       or PACKAGE_RE.search(words) is not None)
170      wants_compress = COMPRESS_RE.search(words) is not None
171
172      steps: list[Step] = []
173      packed = None
174      if wants_archive:
175          fmt = (archive_format_for(named_out) if named_archive else None) or               ("zip" if
ZIP_FORMAT_RE.search(words) else algorithm or ("gz" if level == "fast" else "xz"))
176          packed = named_out if named_archive else str(_unique(source_abs.parent / (source_abs.name +
EXTENSIONS[fmt])))
177          steps.append(Step("create_archive", {"source_path": source, "output_path": packed,
178                                               "compression": fmt, "level": level},
179                            f"bundle into one {fmt} archive ({level} compression)"))
180      elif wants_compress:
181          algo = algorithm or ("gz" if level == "fast" else "xz")
182          packed = named_out or str(_unique(Path(str(source_abs) + "." + algo)))
183          steps.append(Step("compress_file", {"source_path": source, "output_path": packed,
184                                              "algorithm": algo, "level": level},
185                            f"compress with {algo} ({level} level)"))
186      if wants_encrypt:
187          enc_source = packed or source
188          enc_out = gpg_out or (named_out if packed is None else None) or
str(_unique(Path(str(_abs(enc_source)) + ".gpg")))
189          steps.append(Step("encrypt_file", {"source_path": enc_source, "output_path": enc_out},
190                            "encrypt with GPG AES-256"))
```

*Listing B.2: How one sentence becomes an archive step and an encryption step. Source: smart_archive_agent/planner.py, lines 166–190.*

### Tool layer: the one checked entry point (Section 3.2.3)

```
def execute_tool(name: str, args: dict, passphrase: str | None = None) -> dict:
    """Run one tool call. Never raises: failures come back as {'success': False, ...}.

    `name` may carry the TOOL_PREFIX; `args` may be a dict or the JSON string an LLM API returns.
    """
    if isinstance(name, str) and name.startswith(TOOL_PREFIX):
        name = name[len(TOOL_PREFIX):]
    if isinstance(args, str):
        try:
            args = json.loads(args)
        except ValueError:
            return {"success": False, "operation": name, "error": "arguments are not valid JSON",
                    "message": "The tool call was rejected before execution."}
    problems = check_arguments(name, args)
    if problems:
        return {"success": False, "operation": name, "error": "; ".join(problems),
                "message": "The tool call was rejected before execution."}
    try:
        kwargs = dict(args)
        if name in NEEDS_PASSPHRASE:
            kwargs["passphrase"] = passphrase or ask_passphrase(name)
        return FUNCTIONS[name](**kwargs)
    except Exception as exc:
        return {"success": False, "operation": name, "error": str(exc),
                "message": "The operation could not be completed. Review the error and try again."}
```

*Listing B.3: execute_tool() checks every call and never raises an exception. Source: smart_archive_agent/tools/registry.py, lines 128–152.*

### Zip Slip check before anything is written (Section 3.3.2)

```
    root = destination.resolve()

    def check(name: str) -> None:
        target = (root / name).resolve()
        if target != root and root not in target.parents:
            raise ValidationError("Archive contains a path that escapes the destination.")

    # Every member is checked before the destination is created or anything is written.
    if zipfile.is_zipfile(archive):
        with zipfile.ZipFile(archive) as zf:
            for name in zf.namelist():
                check(name)
            destination.mkdir(exist_ok=True)
            zf.extractall(destination)
    else:
        with tarfile.open(archive, "r:*") as tf:
            for member in tf.getmembers():
                check(member.name)
                if member.issym() or member.islnk():
                    raise ValidationError("Archive contains links; extraction stopped for safety.")
```

*Listing B.4: Every archive member is checked before the destination folder is created. Source: smart_archive_agent/tools/archive.py, lines 139–158.*

**Passphrase handling for GPG (Section 4.2, Step 4)**

```
 8  GPG_BASE = ["gpg", "--batch", "--yes", "--no-symkey-cache", "--pinentry-mode", "loopback",
 9              "--passphrase-fd", "0"]
10
11  def _gpg(args: list[str], passphrase: str, failure: str) -> None:
12      try:
13          subprocess.run(GPG_BASE + args, input=passphrase + "\n", check=True,
14                         capture_output=True, text=True, timeout=600)
15      except FileNotFoundError as exc:
16          raise RuntimeError("GPG is not installed. Install it with: sudo apt install gnupg") from exc
17      except subprocess.CalledProcessError as exc:
18          lines = (exc.stderr or exc.stdout or "").strip().splitlines()
19          detail = lines[-1].replace("gpg: ", "", 1) if lines else ""  # Python 3.8: no removeprefix
20          if "bad session key" in detail.lower() or "bad passphrase" in detail.lower():
21              detail = "wrong passphrase"
22          raise RuntimeError(f"{failure} ({detail})" if detail else failure) from exc
```

*Listing B.5: The passphrase reaches gpg on standard input, never as an argument. Source: smart_archive_agent/tools/encryption.py, lines 8–22.*

## B.4 A Tool Definition as the LLM Sees It

This is what get_tools("openai", prefixed=True) returns for our first tool, exactly what the Agentic Shell's LLM receives (Section 4.2, Step 3):

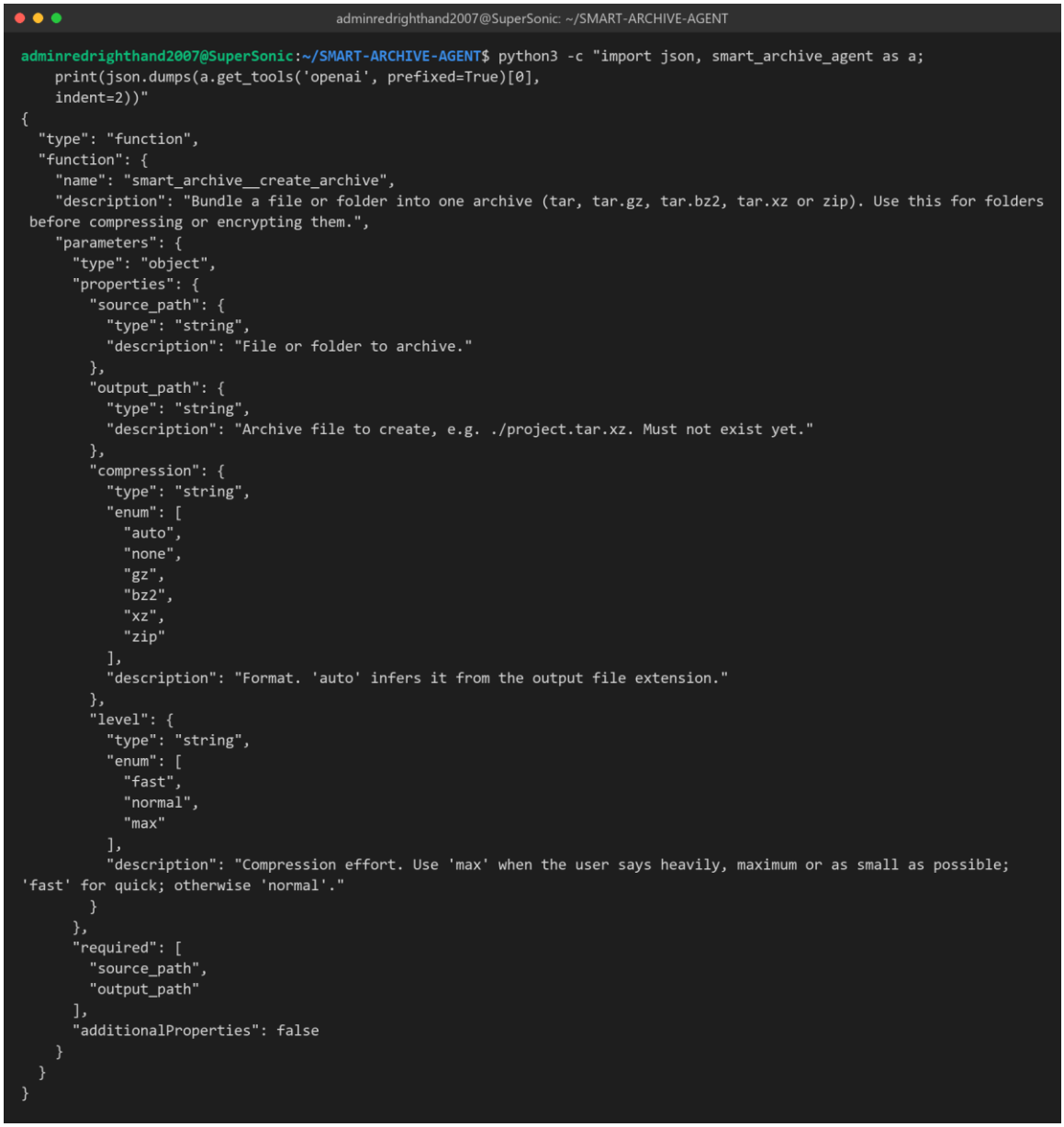



**Figure B.2: Our create_archive tool definition in OpenAI-compatible function-calling format.**

## B.5 The Plan Our Agent Builds for the Brief's Request

Here we only asked the planner for a plan, without running it. The brief's sentence becomes three steps at level max, and the encryption step uses the archive step's output as its input.

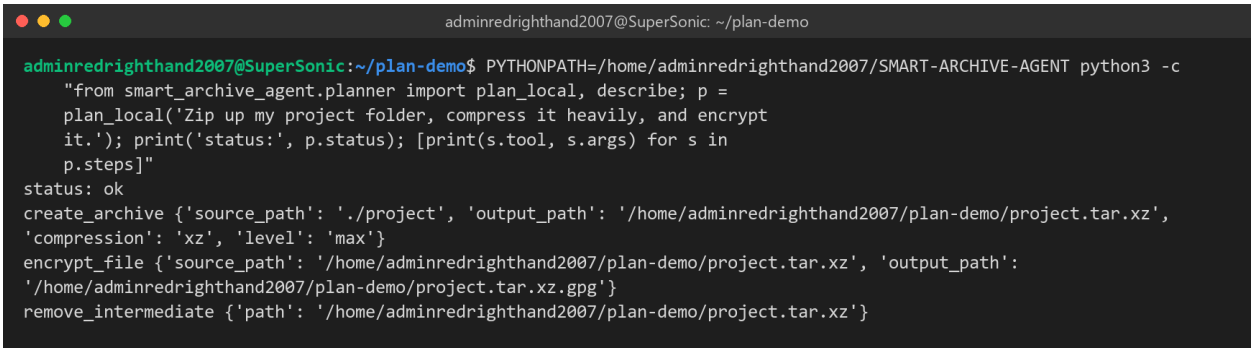



**Figure B.3: plan_local() output for "Zip up my project folder, compress it heavily, and encrypt it."**

## B.6 Running Our Integration Example

examples/agentic_shell_example.py acts like the base shell: it loads our agent, offers its tools, routes the request, runs two LLM-style tool calls, and then tries the offline planner path:

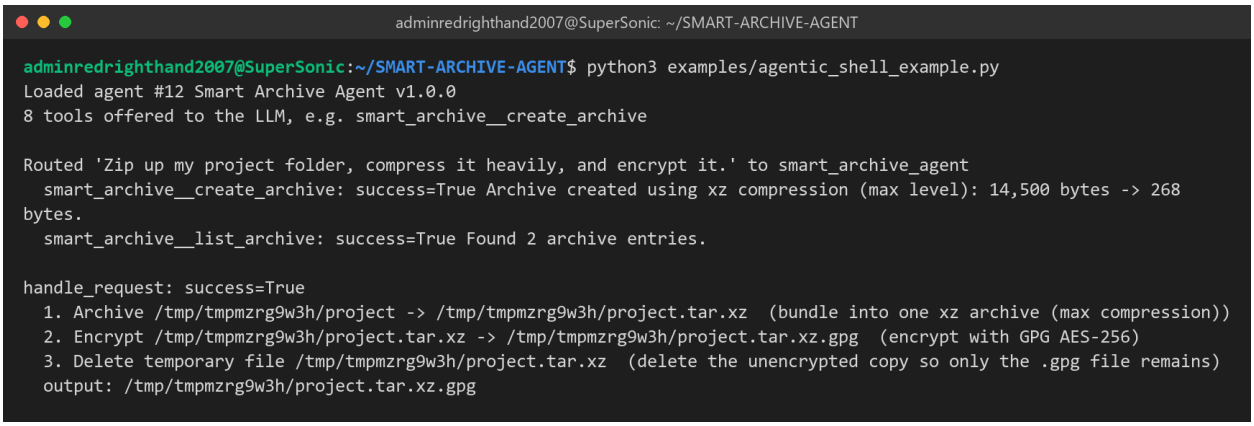



**Figure B.4: Real output of our integration example.**